

NTC_CC105 – DATA STRUCTURE W/LAB

WEEK 5 LABORATORY INSTRUCTION

Laboratory Activity 5

Instructor	Prof. Darwin C. Vargas, MIT
Program	BSIT
Academic Year	2026–2027
Programming Language	C++

1. Student Name	Quintela, Nicole M. / Serdenia Rihanna Shealya
2. Student Number	425001249 / 425002643
3. Section	2.4 BSIT
4. Date	Oct 5, 2026
5. Laboratory Activity No.	Laboratory Activity 5
6. Laboratory Title	Linear Architecture Synthesis & Peer Code Audit

1. Feature: Global State

- BEFORE:

```
int* dataArray = NULL;
int currentCount = 0;
int maxCapacity = 5;
```

- PROBLEM IDENTIFIED: Mutable global variables could be changed by any function, which prevents multiple array instances and breaks modularity.

- AFTER:

```
class DynamicArray
{
private:
    int *data;
    int size;
    int capacity;
    void resize();
public:
    DynamicArray();
    ~DynamicArray();
    ...
};
```

- JUSTIFICATION: Encapsulation makes the data private, so only the class's own member functions can change it. Each object manages its own buffer.

2. Feature: Dynamic Memory Array Resizing

- BEFORE:

```
int* temp = new int[maxCapacity * 2];
for (int i = 0; i < maxCapacity; i++) { temp[i] = dataArray[i]; }
dataArray = temp; // Memory leak!
```

- PROBLEM IDENTIFIED: The old buffer was never freed, so every resize leaked heap memory.

- AFTER:

```
void DynamicArray::resize()
{
```

```

int newCapacity = capacity * 2;
int *temp = new int[newCapacity];
for (int i = 0; i < size; i++)
{
    temp[i] = data[i];
}
delete[] data;
data = temp;
capacity = newCapacity;
}

```

- JUSTIFICATION: The new buffer is allocated, the elements are copied, the old buffer is released with `delete[]`, and only then is the pointer reassigned. This prevents the leak. The copy loop runs to `size` (the elements actually stored) instead of the whole old capacity.

3. Feature: Program Cleanup (RAII)

- BEFORE: // Missing `delete[] dataArray!` Memory Leak on Exit! before `return 0;`
- PROBLEM IDENTIFIED: The heap memory was never released when the program ended.
- AFTER:

```

DynamicArray::DynamicArray()
{
    capacity = 5;
    size = 0;
    data = new int[capacity];
}

```

```

DynamicArray::~~DynamicArray()
{
    delete[] data;
}

```

- JUSTIFICATION: The constructor allocates the buffer, and the destructor runs automatically when records goes out of scope at the end of `main()`. Cleanup no longer depends on the programmer remembering it.

4. Feature: Initialization and Lazy Allocation

- BEFORE:

```

if (currentCount == 0) { dataArray = new int[maxCapacity]; }

```

- PROBLEM IDENTIFIED: `add_item()` allocated a new array whenever the count was 0, so after removing every item the next add would leak the existing buffer. It also left the pointer as NULL until the first insert.
- AFTER: the constructor above allocates the buffer once, and `add()` never reallocates except through `resize()`.
- JUSTIFICATION: One allocation point removes this second leak path and guarantees data is always a valid pointer.

5. Feature: Traversal / Display

- BEFORE: `for (int i = 0; i <= currentCount; i++)`
- PROBLEM IDENTIFIED: The `<=` condition reads one element past the last valid index and prints a garbage value.
- AFTER:

```
void DynamicArray::printAll()
{
    cout << "Current List: ";
    if (size == 0)
    {
        cout << "(empty)";
    }
    for (int i = 0; i < size; i++)
    {
        cout << data[i] << " ";
    }
    cout << endl;
}
```

- JUSTIFICATION: The strict `<` bound visits only indices 0 to `n-1`, so no out-of-bounds read happens. The empty-array message handles the `n = 0` case cleanly.

6. Feature: Upper Bound Validation

- BEFORE: `if (idx < 0) { ... }` with no upper check.
- PROBLEM IDENTIFIED: `remove_item_at(99)` shifted invalid memory and decremented the count, which corrupted the array state.
- AFTER:

```
if (index < 0 || index >= size)
{
    cout << "Invalid index!" << endl;
}
```

```
    return false;
}
```

- JUSTIFICATION: Enforcing $0 \leq \text{index} < \text{size}$ makes invalid requests fail safely with no state change. The function returns `false` so the caller can detect the failure.

7. Feature: Array Deletion Shifting Logic

- BEFORE:

```
for (int i = idx; i < currentCount - 1; i++) {
    for (int j = i; j < currentCount - 1; j++) {
        dataArray[j] = dataArray[j + 1]; break;
    }
}
```

- PROBLEM IDENTIFIED: The inner loop's break meant it ran only once per outer iteration, so it was redundant and unreadable.

- AFTER:

```
for (int i = index; i < size - 1; i++)
{
    data[i] = data[i + 1];
}
size--;
```

- JUSTIFICATION: One clear single-pass loop with $O(n)$ worst case. The asymptotic complexity is unchanged, but the constant-factor overhead and the confusing structure are gone.

8. Feature: Linear Search

- BEFORE: a `for (int k = 0; k < 1; k++)` loop nested inside the search loop.
- PROBLEM IDENTIFIED: The dummy inner loop runs exactly once, so it adds overhead and does nothing useful.
- AFTER:

```
int DynamicArray::find(int value)
{
    for (int i = 0; i < size; i++)
    {
        if (data[i] == value)
        {
            return i;
        }
    }
}
```

```
    }  
  }  
  return -1;  
}
```

- JUSTIFICATION: A single-pass scan with $O(1)$ best case and $O(n)$ worst case. It returns -1 when the value is absent, and it works safely on an empty array.